

Skip List - Αποτελεσματική Αναζήτηση, Εισαγωγή και Διαγραφή σε Συνδεδεμένες Λίστες

Μια λίστα παράλειψης (skip list) είναι μια δομή δεδομένων που επιτρέπει αποτελεσματική αναζήτηση, εισαγωγή και διαγραφή στοιχείων σε μια ταξινομημένη λίστα.

Πρόκειται για πιθανοκρατική δομή δεδομένων, πράγμα που σημαίνει ότι η μέση πολυπλοκότητά της προσδιορίζεται μέσω πιθανοτικής ανάλυσης.

Πώς λειτουργεί μια Skip List;

Σε μια skip list, τα στοιχεία οργανώνονται σε επίπεδα.

- Κάθε επίπεδο έχει λιγότερα στοιχεία από αυτό που βρίσκεται ακριβώς κάτω του.
- Το κατώτερο επίπεδο είναι μια κανονική συνδεδεμένη λίστα.
- Τα ανώτερα επίπεδα περιέχουν συνδέσμους παράκαμψης, που επιτρέπουν ταχύτερη μετάβαση σε απομακρυσμένα στοιχεία της κάτω λίστας.

Η ιδέα είναι να μειώσουμε τα βήματα που χρειάζονται για να φτάσουμε στο

Τεχνική "Ρίψης Κέρματος" (Coin Flipping Technique):

Η τεχνική "ρίψης κέρματος" χρησιμοποιείται για να καθορίσει τυχαία τον αριθμό των επιπέδων που θα καταλάβει ένα στοιχείο σε μια skip list. Όταν εισάγεται ένα νέο στοιχείο:

- Παράγεται ένας τυχαίος αριθμός και συγκρίνεται με ένα προκαθορισμένο όριο.
- Αν ο αριθμός είναι μικρότερος από το όριο, το στοιχείο τοποθετείται στο επόμενο επίπεδο.
- Η διαδικασία επαναλαμβάνεται μέχρι ο τυχαίος αριθμός να ξεπεράσει το όριο, και τότε το στοιχείο τοποθετείται στο χαμηλότερο επίπεδο.

Αυτή η τεχνική είναι αυτή που δίνει στη skip list τον πιθανοκρατικό χαρακτήρα της και εξασφαλίζει τη χαμηλή μέση χρονική πολυπλοκότητα.

Λειτουργία Αναζήτησης (Search Operation):

Η αναζήτηση σε skip list γίνεται από το πάνω προς το κάτω επίπεδο.

- Παρακάμπτουμε στοιχεία που δεν μας ενδιαφέρουν.
- Συνεχίζουμε την αναζήτηση κατεβαίνοντας επίπεδα μέχρι:
 - Να βρούμε το ζητούμενο στοιχείο ή
 - Να διαπιστώσουμε ότι δεν υπάρχει στη λίστα.

Οι σύνδεσμοι "παράκαμψης" (skip links) επιτρέπουν γρήγορη μετάβαση, μειώνοντας τον μέσο αριθμό βημάτων για να φτάσουμε στο στοιχείο.

🕒 Η μέση χρονική πολυπλοκότητα αναζήτησης είναι $O(\log n)$, όπου n είναι το πλήθος των στοιχείων στο κατώτερο επίπεδο.

Λειτουργία Εισαγωγής (Insertion Operation):

Η διαδικασία εισαγωγής ενός νέου στοιχείου σε μια skip list περιλαμβάνει τα παρακάτω βασικά βήματα:

Καθορισμός Αριθμού Επιπέδων (Συνδέσμων) του Νέου Κόμβου

- Κάθε κόμβος στη skip list έχει τουλάχιστον έναν σύνδεσμο (στο χαμηλότερο επίπεδο).
- Ο αριθμός των επιπέδων (δηλαδή πόσους "ορόφους" θα καταλαμβάνει ο κόμβος) καθορίζεται τυχαία με μια διαδικασία που βασίζεται στην πιθανότητα.

💡 Ιδέα:

- Αν στο επίπεδο 1 (χαμηλότερο) υπάρχουν t κόμβοι, τότε θέλουμε:
 - Στο επίπεδο 2: περίπου 1 στους t κόμβους να έχει 2 συνδέσμους,
 - Στο επίπεδο 3: περίπου 1 στους t^2 κόμβους να έχει 3 συνδέσμους, κ.ο.κ.
- Δηλαδή, ένας στους t^i κόμβους να έχει τουλάχιστον $j + 1$ συνδέσμους.

✳️ Αυτό διατηρεί μια ισορροπημένη δομή με ύψος περίπου λογαριθμικό, δηλαδή $\log n$ επιπέδων.

Χρήση της Συνάρτησης `randX()`

- Για να επιτευχθεί η παραπάνω κατανομή, χρησιμοποιείται η συνάρτηση $\text{randX}()$, η οποία επιστρέφει έναν αριθμό i με πιθανότητα:

$$P(i) = 1/2^i$$

- Δηλαδή:
 - Με πιθανότητα $1/2$ θα πάρουμε επίπεδο 1,
 - Με πιθανότητα $1/4$ θα πάρουμε επίπεδο 2,
 - Με πιθανότητα $1/8$ θα πάρουμε επίπεδο 3,
 - κ.ο.κ.

Έτσι, τα περισσότερα στοιχεία έχουν λίγα επίπεδα, και λίγα "τυχερά" στοιχεία έχουν πολλά επίπεδα (λειτουργούν σαν shortcuts για ταχύτερη αναζήτηση).

Δημιουργία και Εισαγωγή του Νέου Κόμβου

- Αφού καθοριστεί η τιμή i από τη $\text{randX}()$, δημιουργείται ένας νέος κόμβος με i συνδέσμους (επίπεδα).
 - Η εισαγωγή του γίνεται όπως και η αναζήτηση:
 - Ξεκινάμε από την κορυφή της λίστας,
 - Κατεβαίνουμε επίπεδο-επίπεδο,
 - Στο κάθε επίπεδο, όταν βρούμε τη σωστή θέση, εισάγουμε τον νέο κόμβο συνδέοντάς τον πριν από τον πρώτο κόμβο που έχει μεγαλύτερο ή ίσο κλειδί.
-

Τι Διασφαλίζουμε:

- Πριν από την εισαγωγή, βεβαιωνόμαστε ότι:
 - Ο κόμβος πριν από το σημείο εισαγωγής έχει κλειδί μικρότερο από αυτό που εισάγουμε.
 - Ο κόμβος μετά από το σημείο εισαγωγής (σε κάθε επίπεδο) έχει κλειδί ίσο ή μεγαλύτερο.

🕒 Η μέση πολυπλοκότητα της εισαγωγής είναι $O(\log n)$.

Λειτουργία Διαγραφής (Deletion Operation):

Η διαγραφή περιλαμβάνει:

- Εύρεση του στοιχείου προς διαγραφή.
- Αφαίρεσή του από όλα τα επίπεδα στα οποία εμφανίζεται.

Η ταξινόμηση της λίστας διατηρείται και μετά τη διαγραφή.

🕒 Μέση πολυπλοκότητα: $O(\log n)$

Ανάλυση Πολυπλοκότητας (Complexity Analysis):

- Οι μέσοι χρόνοι για αναζήτηση, εισαγωγή και διαγραφή είναι: $O(\log n)$
- Αυτό οφείλεται στη χρήση της τεχνικής ρίψης κέρματος, η οποία εξασφαλίζει γρήγορη πρόσβαση και λογαριθμική συμπεριφορά.

Όμως, η χειρότερη περίπτωση μπορεί να είναι $O(n)$: Αυτό συμβαίνει σπάνια, όταν όλα τα στοιχεία εισάγονται μόνο στο κατώτερο επίπεδο.

Πολυπλοκότητα

Οι Skip Lists έχουν:

- Μέσο χρόνο για αναζήτηση: $O(\log n)$
- Μέσο χρόνο για εισαγωγή: $O(\log n)$
- Μέσο χρόνο για διαγραφή: $O(\log n)$

Αυτό είναι παρόμοιο με τις πολυπλοκότητες των ισορροπημένων δέντρων (όπως τα AVL ή τα κόκκινο-μαύρα δέντρα), αλλά με πιο απλή υλοποίηση και χαμηλότερη επιβάρυνση.

Ερώτηση: Μπορούμε να κάνουμε αναζήτηση σε ταξινομημένη συνδεδεμένη λίστα πιο γρήγορα από $O(n)$;

Η απάντηση είναι όχι αν χρησιμοποιήσουμε μια απλή συνδεδεμένη λίστα: Η αναζήτηση γίνεται γραμμικά, άρα η χειρότερη περίπτωση είναι $O(n)$.

- Σε ισορροπημένα δέντρα αναζήτησης, μπορούμε να αγνοήσουμε σχεδόν τα μισά στοιχεία μετά από κάθε σύγκριση.
- Σε ταξινομημένους πίνακες, μπορούμε να κάνουμε δυαδική αναζήτηση, επειδή έχουμε τυχαία προσπέλαση.

Άρα, μπορούμε να ενισχύσουμε τις συνδεδεμένες λίστες με "παραιλήψεις"; Η απάντηση είναι Skip List.

Παράδειγμα Skip List με 2 επίπεδα

- Έχουμε 16 κόμβους.
- Το ανώτερο επίπεδο είναι σαν "λωρίδα ταχείας κυκλοφορίας" (express lane).
- Το κατώτερο επίπεδο είναι σαν "κανονική λωρίδα" που συνδέει κάθε στοιχείο.

Αν θέλουμε να αναζητήσουμε το 50:

1. Ξεκινάμε από τον πρώτο κόμβο του express lane.
2. Κινούμαστε προς τα δεξιά μέχρι να βρούμε κόμβο του οποίου ο επόμενος έχει τιμή > 50 (π.χ. το 30).
3. Κατεβαίνουμε στο "normal lane" και κάνουμε γραμμική αναζήτηση από εκεί και πέρα μέχρι να βρούμε το 50.

Πολυπλοκότητα Αναζήτησης με 2 Επίπεδα

- Αν έχουμε n κόμβους στο κάτω επίπεδο, και $\ln$ στο πάνω:
 - Κάθε τμήμα μεταξύ δύο express κόμβων έχει περίπου $\ln$ κόμβους.

Άρα, η αναζήτηση χρειάζεται:

- $O(\ln)$ κόμβους στο express επίπεδο
- $O(\ln)$ στο normal επίπεδο

Συνολικά: $O(\ln)$ χρόνος

Χρειάζεται $O(\ln)$ επιπλέον μνήμη, αλλά επιταχύνουμε σημαντικά την αναζήτηση.

Πλεονεκτήματα της Skip List

- Αξιόπιστη και σταθερή
- Εισάγουμε νέο κόμβο πολύ γρήγορα
- Πιο εύκολη υλοποίηση από hash tables και δέντρα
- Όσο περισσότεροι κόμβοι υπάρχουν, τόσο λιγότερες πιθανότητες για κακή περίπτωση
- $O(\log n)$ χρόνος για όλες τις βασικές πράξεις
- Η αναζήτηση είναι απλή και κατανοητή

Μειονεκτήματα της Skip List

- Χρειάζεται περισσότερη μνήμη από ισορροπημένα δέντρα
- Δεν υποστηρίζει αναζήτηση προς τα πίσω
- Η αναζήτηση μπορεί να είναι πιο αργή από μια απλή λίστα για μικρά δεδομένα
- Δεν εκμεταλλεύεται σωστά την cache λόγω έλλειψης συνεχούς μνήμης (π.χ. πίνακες)

Μπορούμε να το κάνουμε ακόμα καλύτερα;

Ναι.

Η πολυπλοκότητα της skip list μπορεί να βελτιωθεί με την προσθήκη περισσότερων επιπέδων.

- Έτσι, οι πράξεις αναζήτησης, εισαγωγής και διαγραφής μπορούν να επιτευχθούν με $O(\log n)$ κατά μέσο όρο, χρησιμοποιώντας $O(n)$ επιπλέον μνήμη.









